

[Chap 3] 데이터 처리와 수치 연산 기초



Pandas 설치

- Pandas(판다스) 패키지
 - 데이터를 파일에서 읽어들이고 각종 처리를 수행하며 파일로 써넣는 등의 기능
 - 외부 패키지로서, 별도 설치가 필요
 - 설치 명령어 : pip install OOO
- Pandas의 설치
 - cmd(명령 프롬프트)를 실행 후 'pip install pandas'
 - MacOS에서는 터미널 실행 후 pip3 install pandas

```
cmd. 명령 프롬프트
Microsoft Windows [Version 10.0.19044.1826]
(c) Microsoft Corporation. All rights reserved.

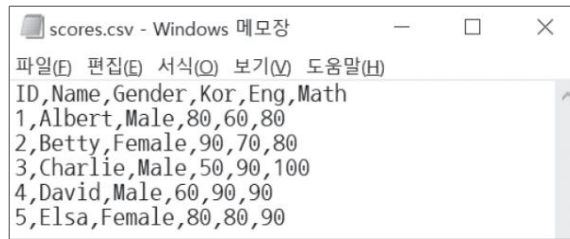
C:\Users\seoyw>pip install pandas_
```

- 파이썬 프로그램에서 외부 패키지를 사용하도록 불러오기

```
문법 import ... as문
import(패키지명) as (별명)
```

Pandas 기초 예제

- 엑셀(.csv) 형태의 데이터 파일 준비



- 파이썬에서 scores.csv를 읽어들이어 출력하기

예제 ex3-1.py

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 print(data)
```

- (1행) pandas를 pd라는 별명으로 불러오기
- (2행) pd.read_csv(파일이름): csv 파일을 불러오는 명령
- 불러온 변수 data는 데이터프레임(dataframe)이라는 자료구조를 가짐
 - 엑셀의 워크시트와 비슷

3

데이터 추출

- 데이터프레임에서 특정 조건을 만족하는 행을 추출

문법 데이터프레임에서 특정 조건을 만족하는 행을 추출하기 위한 명령어
(데이터프레임 이름).loc[(행 조건문)]

- data에서 국어점수가 80점 이상인 사람만 추출

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 print(data.loc[ data['Kor']>=80 ])
```

- (3행) 기본적으로 data.loc[(조건문)]의 형태를 띠고 있고, (조건문)에 해당하는 조건이 'data['Kor']>=80'인 구조이다

4

데이터 추출 (계속)

- 특정 조건을 만족하는 행의 특정 열을 추출

문법 loc을 사용한 특정 행, 열 선택
(데이터프레임 이름).loc[(행 조건문), (열 이름)]

- 국어 점수가 80점 이상인 사람의 이름만 뽑고 싶다면

```
print(data.loc[ data['Kor']>=80, 'Name'])
```

- 국어 점수가 80점 이상인 학생들의 국어 점수에 보너스로 1점씩을 더해주기

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 print(data)
4 data.loc[ data['Kor']>=80, 'Kor']+=1
5 print('<보너스 점수 반영>')
6 print(data)
```

5

데이터 그룹화

- 데이터를 특정 기준으로 그룹화하여 집계하기

문법 groupby를 이용한 데이터 집계
(데이터프레임 이름).groupby(열이름)

- 각 그룹의 내용 보기 : for문 사용

문법 groups 안에서 각 그룹 확인
for (변수1),(변수2) in (그룹 변수 이름)

- (변수1): 그룹 구분
- (변수2): 해당 그룹 데이터

데이터 그룹화 (계속)

- data를 성별에 따라 그룹화하여 출력하기

예제 ex3-2.py

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 groups = data.groupby('Gender')
4 for g, d in groups:
5     print('성별:', g)
6     print(d)
7     print('-----')
```

7

데이터프레임의 각 행별 처리

- 데이터프레임 행 개수 : len(데이터프레임변수)
- 특정 번호 행 꺼내오기: 데이터프레임변수.iloc[행번호]
- 각 행별로 국어, 영어, 수학점수 평균 계산하기

예제 ex3-3.py

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 for i in range(len(data)):
4     row=data.iloc[i]
5     avg=(row['Kor']+row['Eng']+row['Math'])/3
6     print(avg)
```

8

데이터프레임의 각 행별 처리 (계속)

- 각 행에서 계산한 평균값을 데이터프레임에 추가하기

예제 ex3-4.py

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 print(data)
4 print('-----')
5 avgList = []
6 for i in range(len(data)):
7     row=data.iloc[i]
8     avg=(row['Kor']+row['Eng']+row['Math'])/3
9     avgList.append(avg)
10 print(avgList)
11 print('-----')
12 data['Average']=avgList
13 print(data)
```

9

데이터프레임의 저장

- 평균 열이 추가된 데이터프레임을 새로운 엑셀 파일로 저장

예제 ex3-5.py

```
1 import pandas as pd
2 data = pd.read_csv('scores.csv')
3 print(data)
4 print('-----')
5 avgList = []
6 for i in range(len(data)):
7     row=data.iloc[i]
8     avg=(row['Kor']+row['Eng']+row['Math'])/3
9     avgList.append(avg)
10 print(avgList)
11 print('-----')
12 data['Average']=avgList
13 print(data)
14 data.to_csv('score2.csv')
```

행렬(matrix)

- 1개 이상의 숫자 또는 문자가 직사각형의 형태로 배열된 것
- 행렬 표현의 장점
 - 복잡한 식을 간결하게 표현

$$\begin{array}{l} x + 2y = 5 \\ 2x + y = 4 \end{array} \quad \Rightarrow \quad \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix} \quad \Rightarrow \quad AX = B$$

- (여기서, $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ 를 A로, $\begin{bmatrix} x \\ y \end{bmatrix}$ 를 X로, $\begin{bmatrix} 5 \\ 4 \end{bmatrix}$ 를 나타내었음)

- 역행렬을 사용하여 방정식의 해 계산
 - $AX = B$ 의 양변에 A의 역행렬을 곱하기

$$A^{-1}AX = A^{-1}B$$

$$X = A^{-1}B$$

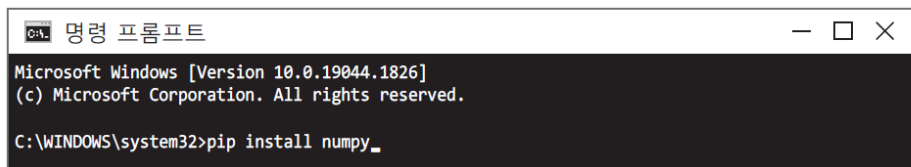
- 행렬 $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ 의 역행렬은 $\begin{bmatrix} -0.33 & 0.67 \\ 0.67 & -0.33 \end{bmatrix}$ 이므로,

$$X = \begin{bmatrix} -0.33 & 0.67 \\ 0.67 & -0.33 \end{bmatrix} \begin{bmatrix} 5 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

11

Numpy(넘파이) 패키지

- 행렬을 생성하고, 값을 수정하거나 계산을 하는 등의 연산을 처리할 수 있는 기능을 제공
- Numpy의 설치
 - 명령 프롬프트를 열어 pip install numpy (MacOS: pip3 install numpy)



- 파이썬 프로그램에 numpy 패키지를 불러들여 사용하기
 - import numpy as np : np라는 이름으로 numpy 패키지를 불러들인다

12

리스트와 numpy 배열

- 리스트를 생성하여 numpy 배열(array)로 변환

예제 ex3-6.py

```
1 import numpy as np
2 a1 = [2,3,5]
3 print(a1)
4 print(type(a1))
5 a2 = np.array(a1)
6 print(a2)
7 print(type(a2))
8 print(a2.shape)
```

- (4, 7행) type(변수명) : 해당 변수의 자료형을 나타냄
- (8행) a2.shape : 해당 변수(a2)의 모양, 즉 행과 열의 개수를 나타냄

```
[2, 3, 5]
<class 'list'>
[2 3 5]
<class 'numpy.ndarray'>
(3,)
```

13

리스트와 numpy 배열 (계속)

- 배열의 특정 값 수정
 - 배열 a2의 첫번째 값을 7로 수정: 리스트와 비슷하게 a2[0]=7

```
1 import numpy as np
2 a1 = [2,3,5]
3 a2 = np.array(a1)
4 print(a2)
5 a2[0]=7
6 print(a2)
```

- Numpy 배열의 장점
 - Numpy의 다양한 기능을 적용 가능
 - 처리 속도가 리스트에 비해 훨씬 빠르고 메모리도 적게 차지함
 - Numpy 배열이 대량 데이터의 수치 연산에 최적화

Numpy 배열의 요소별 연산

- Numpy 배열의 각 요소별 덧셈과 뺄셈

```
1 import numpy as np
2 c = np.array([1,2,3])
3 d = np.array([2,3,5])
4
5 print(c)
6 print(d)
7 print(c+d)
8 print(c-d)
```

```
[1 2 3]
[2 3 5]
[3 5 8]
[-1 -1 -2]
```

15

Numpy 배열의 요소별 연산 (계속)

- Numpy 배열의 각 요소별 곱셈과 나눗셈

```
1 import numpy as np
2 c = np.array([1,2,3])
3 d = np.array([2,3,5])
4
5 print(c*d)
6 print(c/d)
```

```
[2 6 15]
[0.5 0.66666667 0.6]
```


Numpy 배열의 행렬 연산

- 두 행렬의 곱: 왼쪽 행렬의 각 행과 오른쪽 행렬의 각 열이 곱해짐

– 만약 $A = [a, b, c]$ 이고 $B = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ 라면, $AB = ax + by + cz$ 가 되는 방식

- Numpy에서 행렬의 곱은 dot() 함수 사용

```
1 import numpy as np
2 c = np.array([1,2,3])
3 d = np.array([2,3,5])
4
5 print(np.dot(c,d))
6 print(c.dot(d))
```

17

본격적인 행렬 연산

- 2차원 배열(행렬)의 정의

```
1 import numpy as np
2 a = np.array([[2,4,6], [3,5,7]])
3 print(a)
4 print('행렬 형태: ', a.shape)
```

- 특정 행, 열의 요소 출력

```
5 print(a[0,0])
6 print(a[0,2])
7 print(a[1,1])
```

행렬의 요소별 사칙연산

- 행렬에 대해 기본적 사칙연산을 일차원 배열과 마찬가지로 각 요소별로 사칙연산을 적용

예제 ex3-8.py

```
1 import numpy as np
2 a = np.array([[1,2,3],[4,5,6]])
3 b = np.array([[2,4,6],[3,5,7]])
4 print(a+b)
5 print(a-b)
6 print(a*b)
7 print(a/b)
```

```
[[ 3  6  9]
 [ 7 10 13]]
[[-1 -2 -3]
 [ 1  0 -1]]
[[ 2  8 18]
 [12 25 42]]
[[0.5      0.5      0.5      ]
 [1.33333333 1.      0.85714286]]
```

19

행렬곱의 연산

- 행렬의 곱은 numpy에서는 dot() 함수로 표현
 - 왼쪽의 열의 개수와 오른쪽 행의 개수가 맞아야 함
 - 다음 프로그램은 오류 발생
 - a는 3열인데 b는 2행

```
1 import numpy as np
2 a = np.array([[1,2,3],[4,5,6]])
3 b = np.array([[2,4,6],[3,5,7]])
4 print(np.dot(a,b))
```

Traceback (most recent call last):

File "D:/Dropbox/2.Lecture/BusinessAnalytics_2021년_2학기/202108_BA_book/draft_v0.2/BA_code/ch03_Pandas,numpy/ex3-7.py", line 4, in <module>

print(np.dot(a,b))

File "<_array_function__ internals>", line 6, in dot

ValueError: shapes (2,3) and (2,3) not aligned: 3 (dim 1) != 2 (dim 0)

행렬곱의 연산(계속)

- b를 전치(transpose)시켜 3행 2열의 행렬로 변환하여 다시 계산
 - 전치행렬은 np.transpose() 함수 사용

```
1 import numpy as np
2 a = np.array([[1,2,3],[4,5,6]])
3 b = np.array([[2,4,6],[3,5,7]])
4 b = np.transpose(b)
5 print(a)
6 print(b)
7 print(np.dot(a,b))
```

```
[[1 2 3]
 [4 5 6]]
[[2 3]
 [4 5]
 [6 7]]
[[28 34]
 [64 79]]
```

21

단위행렬(identity matrix)

- 대각요소만 1, 나머지는 모두 0인 정방행렬
 - 단위행렬에 어떤 행렬을 곱해도 원래의 행렬이 변하지 않는 특성
 - np.eye(행 개수) 로 생성

```
1 import numpy as np
2 id = np.eye(3)
3 print(id)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

역행렬(inverse matrix)

- 특정한 행렬에 곱했을 때 단위행렬이 되게 하는 행렬
 - `np.linalg.inv()` 사용

```
1 import numpy as np
2 a = np.array([[1,2],[3,4]])
3 b = np.linalg.inv(a)
4 print(a)
5 print(b)
6 print(np.dot(a,b))
```

- 약간의 소수점오차가 발생할 수 있음
 - $8.8817842e-16 = 8.8817842 \times 10^{-16} = 0.000000000000000088817842$ (거의 0)

```
[[1 2]
 [3 4]]
[[-2.  1. ]
 [ 1.5 -0.5]]
[[1.00000000e+00 0.00000000e+00]
 [8.8817842e-16 1.00000000e+00]]
```

23

여러가지 특수행렬의 생성

- 영행렬: 모든 구성요소가 0인 행렬
 - `np.zeros(행 개수, 열 개수)`
 - 행렬 초기화에 자주 사용

```
1 import numpy as np
2 zero = np.zeros((3,5))
3 print(zero)
```

- 기타 특수행렬
 - `np.ones(행 개수, 열 개수)`: 모든 구성요소가 1인 행렬
 - `np.full((행 개수, 열 개수), 초기화 값)`: 모든 구성요소가 초기화 값인 행렬

데이터의 통계치 추출

- 평균, 표준편차, 중앙값 등
 - 각각 `np.mean()`, `np.stdev()`, `np.median()`으로 계산

예제 ex3-9.py

```
1 import numpy as np
2 a = np.array([1,2,3,5,9])
3 print('평균: ', np.mean(a))
4 print('표준편차: ', np.std(a))
5 print('중앙값: ', np.median(a))
6 print('최소값: ', np.min(a))
7 print('최대값: ', np.max(a))
```